Système d'exploitation : TD4, sémaphores

février 2026 — Pierre Rousselin

Remarque : par soucis de concision, les inclusions des fichiers d'en-tête et certains tests d'appels système sont omis dans les programmes présentés dans ce sujet.

Exercice 1 : Exemples de sémaphores

Dans toutes les questions qui suivent, on suppose donnée la fonction `main`, les globales suivantes, et les déclarations suivantes :

```
void *hop(void *);
void *plop(void *);
void travailler(void);
sem_t sem1;
sem_t sem2;
int res;
int main(void)
{
        pthread_t th_hop, th_plop;
        sem_init(&sem1, 0, X);
        sem_init(&sem2, 0, Y);
        pthread_create(&th_hop, NULL, hop, NULL);
        pthread_create(&th_plop, NULL, plop, NULL);
        pthread_join(th_hop, NULL);
        pthread_join(th_plop, NULL);
        sem_destroy(&sem1);
        sem_destroy(&sem2);
        return 0;
}
```

Les valeurs initiales `X` et `Y` des sémaphores seront données pour chaque question. La fonction `travailler` représente un travail, arbitraire, qui ne modifie ni les sémaphores, ni la variable `res`. Donner le comportement du programme ou, si une situation de compétition ou une étreinte mortelle (*deadlock*, le programme est bloqué) sont possibles.

1. `X = 0`, `Y = 1` et les fonctions `hop` et `plop` sont les suivantes :

```
void *hop(void *arg) {
        travailler();
        sem_wait(&sem2);
        res = 42;
        sem_post(&sem1);
        travailler();
        sem_wait(&sem2);
        res = 93;
        sem_post(&sem1);
        return NULL;
}
void *plop(void *arg) {
        travailler();
        sem_wait(&sem1);
        printf("%d\n", res);
        sem_post(&sem2);
        travailler();
        sem_wait(&sem1);
        printf("%d\n", res);
        sem_post(&sem2);
        return NULL;
}
```

2. Mêmes fonctions avec `X = 1` et `Y = 1`.

3. Mêmes fonctions avec `X = 0` et `Y = 0`.

4. Mêmes fonctions avec `X = 0` et `Y = 2`.

5. Les fonctions `hop` et `plop` sont les suivantes, `X = 1` et `sem2` n'est pas utilisé. La fonction `main` affiche le contenu de `res` après avoir `join` les deux *threads*. Attention, l'incrémentation n'est pas supposée atomique.

```
void *hop(void *arg) {
        for (int i = 0; i < 3; i++) {
                sem_wait(&sem1);
                res++;
                sem_post(&sem1);
        }
        return NULL;
}
void *plop(void *arg) {
        return hop(NULL);
}
```

6. Même chose pour `X = 2`.

7. Même chose pour `X = 0`.

--- * ---

Exercice 2 : A, puis B

On considère le programme suivant :

```c
void boulot(int n);
int main(void) {
        boulot(100000);
        printf("A\n");
        boulot(100000);
        printf("B\n");
}

void boulot(int n) {
        for (int i = 0; i < n; i++)
                ;
}
```

1. Faire faire le `boulot` à des *threads* :
   — un nouveau *thread* fait le boulot puis quand il a fini écrit `A` sur le terminal ;
   — un autre nouveau *thread* fait le boulot puis quand il a fini écrit `B` sur le terminal.
2. Quel est le risque si le *thread* initial n'attend pas les autres ?
3. L'ordre des affichages est-il fixé à l'avance ?
4. À l'aide d'un sémaphore, faire en sorte que `A` soit toujours affiché avant `B`. Attention, ces *threads* doivent pouvoir travailler en même temps !
5. Reprendre l'exercice avec trois `boulots` et l'écriture de `A`, puis `B`, puis `C`, dans cet ordre.

--- * ---

Exercice 3 : interblocage (*deadlock*)

On considère le programme (idiot) suivant :

```c
void *f1(void *arg);
void *f2(void *arg);
int main(void)
{
        pthread_t th1, th2;
        sem_t sem[2];
        sem_init(sem, 0, 1);
        sem_init(sem + 1, 0, 1);
        pthread_create(&th1, NULL, f1, sem);
        pthread_create(&th2, NULL, f2, sem);

        pthread_join(th1, NULL);
        pthread_join(th2, NULL);
        sem_destroy(sem);
        sem_destroy(sem + 1);

        return 0;
}
void *f1(void *arg)
{
        sem_t *sem = arg;
        sem_wait(sem);
        printf("f1: sem[0] mis à 0\n");
        sem_wait(sem + 1);
        printf("f1: sem[1] mis à 0.\n");
        sem_post(sem + 1);
        sem_post(sem);
        return NULL;
}
void *f2(void *arg)
{
        sem_t *sem = arg;
        sem_wait(sem + 1);
        printf("f2: sem[1] mis à 0\n");
        sem_wait(sem);
        printf("f2: sem[0] mis à 0\n");
        sem_post(sem);
        sem_post(sem + 1);
        return NULL;
}
```

Donner les différents scénarios possibles dans l'exécution de ce programme (on suppose que les affichages sont atomiques, ce qui est faux mais simplifie l'exercice).

--- * ---